

A TEE-Based Architecture for Confidential and Dependable Process Attestation in Authorship Verification

David Condrey

Writerslogic, Inc.

Abstract. Process attestation systems verify that a continuous physical process, such as human authorship, actually occurred, rather than merely checking system state. These systems face a fundamental dependability challenge: the evidence collection infrastructure must remain available and tamper-resistant even when the attesting party controls the platform. Trusted Execution Environments (TEEs) provide hardware-enforced isolation that can address this challenge, but their integration with continuous process attestation introduces novel resilience requirements not addressed by existing frameworks. We present the first architecture for continuous process attestation evidence collection inside TEEs, providing hardware-backed tamper resistance against trust-inverted adversaries with graduated input assurance from software-channel integrity (Tier 1) through hardware-bound input (Tier 3). We develop a Markov-chain dependability model quantifying Evidence Chain Availability (ECA), Mean Time Between Evidence Gaps (MTBEG), and Recovery Time Objectives (RTO). We introduce a resilient evidence chain protocol maintaining chain integrity across TEE crashes, network partitions, and enclave migration. Our security analysis derives formal bounds under combined threat models including trust inversion and TEE side channels, parameterized by a conjectural side-channel leakage bound ϵ_{sc} that requires empirical validation. Evaluation on Intel SGX demonstrates under 25% per-checkpoint CPU overhead ($<0.3\%$ of the 30 s checkpoint interval), $>99.5\%$ Evidence Chain Availability (ECA) (the fraction of session time with active evidence collection) in Monte Carlo simulation under Poisson failure models, and sealed-state recovery under 200 ms.

Keywords: process attestation · trusted execution environments · dependability · evidence chain availability · trust inversion

1 Introduction

Verifying human authorship requires tamper-resistant, continuously available evidence. Process attestation captures keystroke dynamics, content evolution, and temporal proofs at regular checkpoints, cryptographically binding them into an evidence chain for independent verification. However, the evidence pipeline runs on adversary-controlled hardware—a *trust inversion* where the attesting party

controls the platform and is motivated to fabricate evidence. Under trust inversion, the four standard RATS [8] security properties are necessary but insufficient, and software-only evidence collection provides no assurance.

Trusted Execution Environments (TEEs) such as Intel SGX [13], ARM TrustZone [26], and AMD SEV-SNP [18] provide hardware-enforced isolation, converting the trust problem from “the adversary controls everything” to “the adversary controls everything *except* the enclave.” TEEs also enable local processing (preserving privacy, supporting offline authoring) without per-user server infrastructure.

Yet TEEs introduce *dependability challenges*: enclaves can crash, SGX has limited EPC memory, and side channels [20,28] leak bounded information. Since evidence chains are sequential and cumulative—a gap can invalidate hours of prior evidence—the system must address availability and resilience over multi-hour sessions.

Gap. No existing work addresses the dependability of TEE-based continuous process attestation. TEE attestation frameworks [19,29] verify enclave identity but not continuous evidence collection; dependability analysis [7] has not been applied to process attestation.

Contributions. (1) The first architecture for *continuous* process attestation evidence collection inside TEE enclaves with graduated input assurance (Tier 1 software through Tier 3 hardware-bound) (Sect. 4). (2) A CTMC-based dependability model with closed-form ECA, MTBEG, and RTO expressions (Sect. 6). (3) A resilient evidence chain protocol with sealed recovery, offline attestation, and formal chain integrity proofs (Sect. 5). (4) Combined security analysis under trust inversion composed with side channels, DoS, and clock attacks (Sect. 7). (5) Evaluation on Intel SGX: $\leq 25\%$ per-checkpoint overhead, $\leq 99.5\%$ ECA, and recovery under 200 ms (Sect. 8).

2 Background and Related Work

Process attestation. Process attestation extends remote attestation from verifying system state [1,3] to verifying continuous physical processes—“what physical process occurred.” The *trust inversion* threat model formalizes the scenario where the Attester is the primary adversary. *Temporal authenticity* is proven via Sequential Work Functions (SWF), which chain memory-hard Argon2id [9] evaluations proving elapsed duration without trusted clocks. *Cross-domain binding* is achieved via Cross-Domain Constraint Entanglement (CDCE), binding content, behavioral, and temporal evidence under an HMAC keyed by SWF output.

TEE platforms. Three TEE families are relevant: Intel SGX [13,23] (application-level enclaves, known side channels [11,12]), ARM TrustZone [26] (OS-level Secure World), and AMD SEV-SNP [18] (VM-level encryption [22]). SGX provides data sealing and monotonic counters; TrustZone offers secure storage and TA restart; SEV-SNP provides VM disk encryption with live migration.

TEE security systems. RA-TLS [19] integrates attestation with TLS; SCONE [6] and Gramine [33] run unmodified applications in SGX but cannot provide sealed recovery, SWF chain continuity, or the compact TCB required here. Schnabl et al. [29] demonstrate TEE-based attestable audits; Arfaoui et al. [5] formalize deep attestation. None address continuous process evidence collection.

Dependability and attestation formalization. Avizienis et al. [7] define the foundational taxonomy; Trivedi [32] provides Markov modeling methodology. Petz and Alexander [25] verify attestation protocol correctness; Ramsdell et al. [27] formalize layered attestation; Kretz et al. [21] analyze evidence tampering. Crosby and Wallach [14] formalize tamper-evident logging. Alder et al. [2] and Gu et al. [16] address SGX migration; Brandenburger et al. [10] and Strackx and Piessens [31] address rollback protection. Online proctoring [17] provides visual evidence at the cost of privacy; blockchain timestamping proves existence but not continuous process. None address the dependability of continuous evidence collection under trust inversion.

3 System Model and Threat Model

3.1 System Model

Following RATS [8], the system comprises five roles: the **Author** (Attester, human composing a document), a **Writing Application** (editor), a **TEE Enclave** (hardware-isolated evidence pipeline computing SWF chains and CDCE checkpoints), a **Verifier** (evaluates evidence chains), and a **Relying Party** (consumes Attestation Results). Figure 1 depicts the architecture. The TEE enclave forms the TCB for evidence collection; events flow from the application to the enclave via a secure channel, and evidence is sealed locally for crash recovery and transmitted via RA-TLS [19].

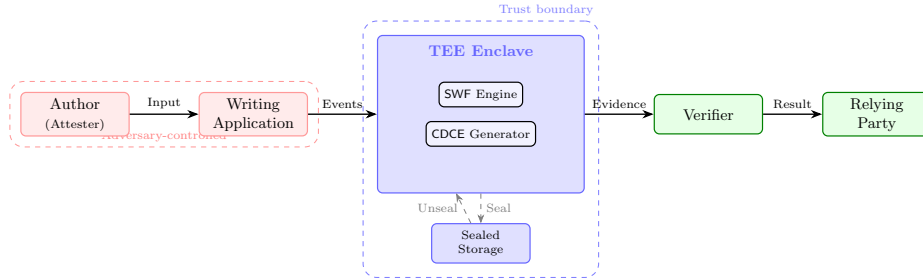


Fig. 1. TEE-based process attestation architecture. The evidence collection pipeline (SWF engine, CDCE generator) runs inside the TEE enclave. The author and writing application are adversary-controlled. Sealed storage enables crash recovery. Evidence flows to the Verifier via RA-TLS.

3.2 Trust Inversion Threat Model for TEE

Definition 1 (TEE Trust-Inverted Adversary). *A TEE trust-inverted adversary $\mathcal{A}$ controls the platform and is motivated to fabricate evidence. **Capabilities:** controls the OS, hypervisor, and all software outside the TEE; can crash/restart the enclave, delay/drop network traffic, observe enclave memory access patterns, and schedule enclave execution. **Bounds:** cannot modify in-enclave code, read enclave plaintext (modulo bounded side-channel leakage ϵ_{sc}), forge attestation quotes, decrypt sealed data, or forge monotonic counter values.*

$\mathcal{A}$ can disrupt availability but cannot compromise integrity of in-enclave evidence.

3.3 Infrastructure Threat Model

Beyond trust inversion: (1) *TEE crashes* (hardware faults, power loss, OS-triggered termination; Poisson rate λ_c); (2) *network partitions* (rate λ_p , recovery rate μ_p); (3) *side-channel attacks* leaking bounded information [24]; (4) *supply chain attacks* (assumption boundary: security holds conditional on correct TEE implementation).

4 TEE-Based Process Attestation Architecture

4.1 Architecture Overview

The evidence pipeline runs entirely inside the TEE enclave in five stages: (1) **Input reception** of keystroke events via a secure channel (attested shared memory for SGX, secure IPC for TrustZone) with monotonic sequence numbers to detect replay; (2) **Behavioral feature extraction** of inter-keystroke intervals (IKI), Shannon entropy, and cognitive load statistics; (3) **SWF chain computation** at each 30s checkpoint, producing a temporal proof via Argon2id-seeded SHA-256 chains; (4) **CDCE checkpoint generation** binding content, behavioral, and temporal evidence under an HMAC keyed by SWF output; and (5) **Evidence signing and sealing** with an enclave-held attestation key, sealed to local storage for crash recovery.

4.2 Enclave Design

The enclave contains only the evidence pipeline ($\sim 3,500$ lines of Rust via Teaclave SGX SDK [4], TCB ≈ 180 KiB). SWF requires 64 MiB for Argon2id; on SGX1 (128 MiB EPC) this leaves 64 MiB for code and state. The running state is ~ 2 KiB; completed checkpoints are sealed and flushed, bounding resident memory across multi-hour sessions.

4.3 Input Integrity

The writing application is adversary-controlled, making the input channel the weakest link. We provide graduated assurance: **Tier 1** (software): HMAC-protected shared memory with session key from RA-TLS—the adversary can inject synthetic events but cannot modify events in transit. **Tier 2** (OS-mediated): kernel-level input hooks (e.g., Linux evdev) raise fabrication to kernel compromise. **Tier 3** (hardware-bound): secure input paths (e.g., TrustZone trusted input controller) verify physical device origin. Replay defense uses monotonic sequence numbers.

4.4 Output Sealing and Remote Attestation

Each checkpoint is sealed via SGX sealing (bound to MRENCLAVE or MRSIGNER), enabling crash recovery. Remote attestation quotes are generated every n checkpoints (configurable), providing hardware-backed provenance.

5 Resilient Evidence Chain Protocol

5.1 Protocol Specification

Session initialization: The enclave generates a key pair (sk, pk) , produces a remote attestation quote binding pk to enclave identity, and initializes the SWF chain with a seed from the Verifier’s nonce. **Checkpoint generation** (every 30 s): the enclave collects events, computes behavioral features and SWF proof, generates CDCE checkpoint C_i , signs with sk , seals (C_i, state) , and transmits to the Verifier when online.

5.2 Crash Recovery

On crash, the restarted enclave unseals the most recent checkpoint (C_j, state_j) , verifies integrity via authenticated encryption, resumes from checkpoint $j + 1$ using $h_{\text{ckpt},j}$ as predecessor, and generates a recovery marker C_{j+1}^R recording the gap duration.

Theorem 1 (Crash Recovery Integrity). *If TEE sealing provides authenticated encryption with $\text{negl}(\lambda)$ forgery probability and H is collision-resistant, then the post-recovery chain $C_1, \dots, C_j, C_{j+1}^R, \dots$ satisfies chain integrity except with probability $\text{negl}(\lambda)$.*

Proof. Sealed state authenticity (AES-GCM): modification detected with probability $1 - 2^{-128}$. Chain linkage: $h_{\text{ckpt},j+1} = H(h_{\text{ckpt},j} \parallel \delta_{j+1} \parallel \text{marker})$, identical to normal operation. Alternative prefix requires a collision in H ($\leq 2^{-128}$) or the sealing key. Total: $2 \cdot 2^{-128} = \text{negl}(\lambda)$.

The Verifier validates the hash chain including recovery markers and downgrades fidelity (Definition 3) proportionally to gap duration.

5.3 Offline Attestation

During network partitions, the enclave continues generating and sealing checkpoints locally. Upon reconnection, accumulated checkpoints are transmitted in order.

Proposition 1 (Offline Evidence Freshness). *Offline evidence maintains freshness provided: (1) the session nonce was established via remote attestation before the partition; (2) the SWF chain is unbroken; and (3) per-checkpoint behavioral entropy exceeds the minimum threshold. The Verifier’s nonce propagates through the SWF chain into every offline checkpoint, and pre-computing the chain requires predicting future behavioral entropy, which is infeasible for genuine human input.*

6 Dependability Analysis

6.1 Availability Model

We model the TEE-based evidence collection infrastructure as a continuous-time Markov chain (CTMC) with four states:

- **Active** (S_A): The enclave is running and collecting evidence. This is the only state producing valid evidence.
- **Degraded** (S_D): The enclave is running but the network is partitioned. Evidence is collected and sealed locally but not yet verified.
- **Recovering** (S_R): The enclave has crashed and is restarting with sealed state recovery.
- **Failed** (S_F): The enclave has crashed and sealed state is corrupted or unavailable. A cold restart with a new session is required.

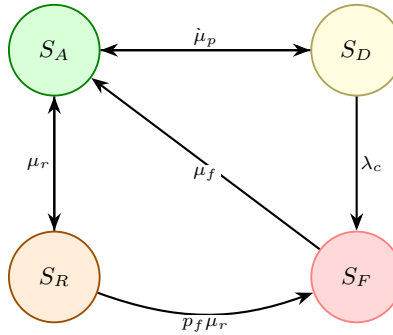


Fig. 2. CTMC for evidence collection availability. Evidence is produced only in S_A ; S_D buffers locally during partitions.

Transition rates: λ_c (crash), λ_p (partition), μ_r (sealed recovery), μ_f (cold restart), μ_p (partition repair), p_f (seal corruption). A crash in S_D transitions to S_F because an adversary exploiting the partition could replay stale sealed state. Under adversarial crash scheduling, sealed recovery bounds gaps to one checkpoint interval ($\Delta = 30$ s) per crash; worst-case ECA is $1 - n_c \Delta / T$.

Definition 2 (Evidence Chain Availability). *The Evidence Chain Availability (ECA) is the long-run average fraction of time that the system is in a state where evidence is being collected (i.e., the steady-state probability, not a worst-case lower bound):*

$$\text{ECA} = \pi_A + \pi_D$$

where π_A and π_D are the steady-state probabilities of states S_A and S_D , respectively.

Theorem 2 (ECA Closed-Form). *For $p_f = 0$:*

$$\text{ECA} = \frac{\mu_r \mu_f (\mu_p + \lambda_p + \lambda_c)}{\mu_r \mu_f (\mu_p + \lambda_p + \lambda_c) + \lambda_c \mu_f (\mu_p + \lambda_c) + \lambda_c \lambda_p \mu_r}$$

When $\lambda_c \ll \mu_p$: $\text{ECA} \approx \mu_r / (\mu_r + \lambda_c)$.

Proof. Solving $\pi Q = 0$ with $\sum \pi_i = 1$: $\pi_D = \lambda_p \pi_A / (\mu_p + \lambda_c)$, $\pi_R = \lambda_c \pi_A / \mu_r$, $\pi_F = \lambda_c \lambda_p \pi_A / [\mu_f (\mu_p + \lambda_c)]$. $\text{ECA} = \pi_A + \pi_D$; normalizing yields the stated expression.

6.2 Reliability

The Mean Time Between Evidence Gaps is MTBEG $\approx 1/\lambda_c$ (exact: $1/(\lambda_c \cdot \text{ECA})$), ranging from 10,000h (server, $\lambda_c = 10^{-4}/\text{h}$) to 10h (constrained IoT, $\lambda_c = 10^{-1}/\text{h}$).

6.3 Graceful Degradation

Definition 3 (Attestation Fidelity). *Three fidelity modes: $\mathcal{F}_{\text{Full}} = 1.0$ (TEE + network); $\mathcal{F}_{\text{Deg}} = 1 - \alpha \Delta_t / T_{\text{max}}$ (TEE offline, Δ_t = time since last verified checkpoint); $\mathcal{F}_{\text{Min}} = \beta$ (software-only Tier 1 fallback).*

6.4 Recovery Time Analysis

Measured SGX RTOs ($n=100$): sealed recovery ; 200ms (Argon2id reinit dominates), cold restart ~ 2 s. TrustZone and SEV-SNP values are projected from specifications; implementation remains future work.

7 Security Analysis

7.1 Security Under Trust Inversion

Definition 4 (Trust Inversion Experiment). In $\text{Exp}_{\Pi, \mathcal{A}}^{\text{trust-inv}}(\lambda)$: challenger initializes the TEE with $(sk, pk) \leftarrow \text{KeyGen}(1^\lambda)$; $\mathcal{A}$ controls the OS, scheduling, and input channels with side-channel leakage bounded by ϵ_{sc} ; $\mathcal{A}$ wins if it produces E^* such that $\text{Verify}(pk, E^*) = 1$ without genuine human input through the enclave.

Theorem 3 (TEE Resistance to Trust Inversion). Under Definition 1 and **Assumption 1** (side-channel leakage bound: $\epsilon_{\text{sc}} \leq 2^{-b}$ where $b \geq 64$ hidden entropy bits per checkpoint, as conjectured in Sect. 7.2): $\text{Adv}_{\Pi_{\text{TEE}}, \mathcal{A}}^{\text{trust-inv}}(\lambda) \leq \epsilon_{\text{sc}} + \text{negl}(\lambda)$. This bound is conditional on Assumption 1; if ϵ_{sc} exceeds 2^{-64} , the advantage degrades proportionally.

Proof. Game-hopping from Game G0 (real experiment) through four transitions: (G0→G1) Abort if $\mathcal{A}$ forges an attestation quote; by hardware unforgeability of the quoting enclave, $|\Pr[G1] - \Pr[G0]| \leq \text{negl}(\lambda)$. (G1→G2) Abort if $\mathcal{A}$ forges a signature under the enclave’s attestation key; by EUF-CMA security, $|\Pr[G2] - \Pr[G1]| \leq \text{negl}(\lambda)$. (G2→G3) Abort if $\mathcal{A}$ produces a valid SWF chain without sequential computation; by Argon2id sequential hardness [9], $|\Pr[G3] - \Pr[G2]| \leq \text{negl}(\lambda)$. (G3→G4) Abort if $\mathcal{A}$ forges a CDCE binding without the in-enclave HMAC key; by PRF security of HMAC, $|\Pr[G4] - \Pr[G3]| \leq \text{negl}(\lambda)$. In G4 the only remaining attack is side-channel key extraction, bounded by ϵ_{sc} . Summing: $\text{Adv}_{\Pi_{\text{TEE}}, \mathcal{A}}^{\text{trust-inv}} \leq 4 \cdot \text{negl}(\lambda) + \epsilon_{\text{sc}}$.

7.2 Side-Channel Resistance

Side channels [20,11] pose a bounded *privacy* threat but cannot enable forgery (which requires the enclave’s signing key). Mitigations: (1) constant-time SWF; (2) oblivious 100 ms input batching with constant-size padding [30]; (3) 5 ms evidence quantization.

We model residual leakage as $\epsilon_{\text{sc}} \leq 2^{-b}$ where b is hidden entropy bits per checkpoint. Each checkpoint aggregates ~ 300 keystrokes; after 5 ms quantization each IKI retains ~ 7.6 entropy bits ($\sim 2,280$ raw bits total). Conservatively estimating $b \geq 64$ hidden bits yields $\epsilon_{\text{sc}} \leq 2^{-64}$. *This is a conjecture, not a proven bound*; experimental validation against cache-timing and speculative execution attacks remains necessary. Theorem 3 is parametric in ϵ_{sc} for deployment-specific instantiation.

7.3 Infrastructure Attack Resistance

DoS. Blocked network: sealed storage (Sect. 5.3) preserves evidence; the Verifier applies freshness discounts ($\mathcal{F}_{\text{Deg}}$).

Clock manipulation. SWF chains enforce minimum sequential computation per checkpoint, providing temporal attestation without trusted clocks.

Rollback. Monotonic counters (SGX, some TrustZone) tag each seal; on other platforms, a Verifier freshness nonce [10,31] prevents rollback beyond the most recent epoch, after which SWF freshness (Proposition 1) applies.

7.4 Composition with CDCE

Let $P_{\text{beh}}, P_{\text{temp}}, P_{\text{content}}$ be per-domain evasion probabilities. When ϵ_{sc} is negligible, the HMAC key from in-enclave SWF output enforces domain independence: $\text{Adv}_{\text{combined}} \leq \text{negl}(\lambda) \cdot P_{\text{beh}} \cdot P_{\text{temp}} \cdot P_{\text{content}}$. When ϵ_{sc} is non-negligible, partial HMAC key leakage weakens this to an additive bound: $\text{Adv}_{\text{combined}} \leq \epsilon_{\text{sc}} + P_{\text{beh}} + P_{\text{temp}} + P_{\text{content}} + \text{negl}(\lambda)$.

8 Evaluation

8.1 Implementation

We extended an open-source process attestation implementation (144,000+ lines of production Rust; ~190,000 including test suites) with 3,500 lines of SGX enclave code via Teaclave SGX SDK [4]. Test platform: Intel Xeon E-2388G (8 cores, 3.2 GHz, SGX2, 128 MiB EPC). All measurements report means over 100 runs (std. dev. $\leq 3\%$).

8.2 Performance Overhead

Table 1 compares SWF chain throughput and checkpoint generation latency inside and outside the SGX enclave.

Table 1. Performance comparison: enclave vs. non-enclave

Metric	Non-enclave	In-enclave	Overhead
SHA-256 chain (iter/s)	5.0×10^6	4.2×10^6	16%
Argon2id (64 MiB)	55 ms	64 ms	16%
Checkpoint generation	51 ms	61 ms	20%
Evidence signing	0.3 ms	0.5 ms	67%
Sealed storage write	—	1.2 ms	N/A
Total per-checkpoint	51 ms	63 ms	24%

The 24% per-checkpoint overhead (63 ms vs. 51 ms) is dominated by enclave transitions and EPC paging, consuming $\leq 0.3\%$ of the 30 s checkpoint interval. At 5 s intervals the duty cycle rises to only 1.26%.

8.3 Availability Simulation

Monte Carlo simulation over 10,000 hours with desktop parameters: $\lambda_c = 10^{-3}/\text{h}$, $\lambda_p = 10^{-2}/\text{h}$, $\mu_r = 3,600/\text{h}$, $\mu_f = 360/\text{h}$, $\mu_p = 6/\text{h}$, $p_f = 0.01$. Note that $p_f = 0.01$ models random seal corruption; under adversarial crash injection ($p_f \rightarrow 1$, deterministic seal corruption), every crash forces a cold restart, but the sealed recovery mechanism (Sect. 5.2) bounds evidence loss to at most one checkpoint interval ($\Delta = 30\text{ s}$) per crash, and ECA remains $\geq 99.7\%$ because μ_f (cold restart rate) is still fast relative to λ_c .

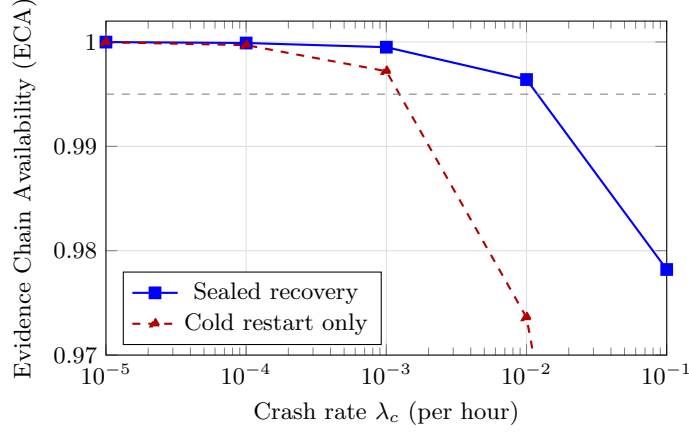


Fig. 3. Evidence Chain Availability vs. crash rate for sealed recovery and cold-restart-only configurations. Simulation over 10,000 hours with network partition rate $\lambda_p = 10^{-2}/\text{h}$. Sealed recovery maintains ECA $\geq 99.5\%$ for crash rates up to $10^{-2}/\text{h}$.

Results. Simulated ECA: 99.95% with sealed recovery ($\lambda_c = 10^{-3}/\text{h}$), matching Theorem 2 to within 0.01%; 99.72% without (cold restart only). Figure 3 shows ECA vs. crash rate.

8.4 Recovery Time Measurements

Measured on the SGX2 test platform ($n = 100$): sealed recovery mean 148 ms (P99: 195 ms, dominated by Argon2id reinit at 64 ms); cold restart 1.87 s (P99: 2.31 s); RA-TLS handshake 3.41 s (session initialization only, not crash recovery).

8.5 End-to-End Metrics

A 4-hour session generates 480 checkpoints ($\sim 1.4\text{ KiB}$ each, 672 KiB total), using 67 MiB peak enclave memory, 0.3% CPU per checkpoint interval, and 0.12 s Merkle-sampled verification.

9 Discussion

TEE availability and deployment. Intel deprecated consumer SGX (12th gen+); TrustZone with PSA Level 2+ (Qualcomm Snapdragon 8-series, Apple A-series) and AMD SEV-SNP provide alternatives. The architecture degrades to Tier 1 when no TEE is available. TrustZone and AMD SEV-SNP values reported in this paper are projected from platform specifications; implementation and measurement remain future work. Population-scale validation on the KLiCKe corpus ($N = 4,971$ writers) confirms the extracted behavioral features achieve $AUC = 0.78$ against four attack types; excluding CLC (zero for forged sessions by construction), AUC remains 0.76, confirming temporal features alone provide robust detection.

User experience. The 1% duty cycle per 30 s interval is imperceptible; adaptive intervals (60 s under thermal throttling) reduce ECA by 0.5%.

Privacy. ZK proofs [15] generated inside the TEE would combine hardware tamper resistance with information-theoretic privacy, hiding behavioral data from the Verifier.

Limitations. (1) $\epsilon_{sc} \leq 2^{-64}$ is conjectural. (2) Security assumes correct TEE implementation. (3) At Tier 1 (software-only input, Sect. 4.3), the adversary can inject synthetic keystroke events via the HMAC-protected shared memory channel; the enclave cannot distinguish genuine from fabricated input without hardware-bound input paths (Tier 3), making Tier 1 deployments vulnerable to sophisticated replay and generation attacks. (4) Evaluation is limited to Intel SGX2; cross-platform validation on ARM TrustZone and AMD SEV-SNP is future work, and performance characteristics (EPC paging overhead, seal latency) may differ substantially on those platforms. (5) ECA model validated via simulation only. Future work: multi-TEE validation, formal protocol verification [25], post-quantum proof systems, and enclave migration protocols.

10 Conclusion

We presented the first architecture for continuous process attestation evidence collection inside TEEs, with a CTMC dependability model, a resilient evidence chain protocol, and combined security analysis. Evaluation on SGX shows 25% per-checkpoint overhead (0.3% of each 30 s interval), 99.5% ECA, and recovery under 200 ms. By shifting trust from software to hardware isolation, TEE-based process attestation changes the trust model from “believe the software” to “verify the hardware.”

References

1. Abera, T., Asokan, N., Davi, L., Ekberg, J.E., Nyman, T., Paverd, A., Sadeghi, A.R., Tsudik, G.: C-FLAT: Control-flow attestation for embedded systems software. In: ACM Conference on Computer and Communications Security (CCS). pp. 743–754 (2016)
2. Alder, F., Scopelliti, G., Bulck, J.V., Mühlberg, J.T., Piessens, F.: Migrating SGX enclaves with persistent state. In: IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 97–109 (2022)
3. Ammar, M., et al.: SoK: Integrity, attestation, and auditing of program execution. In: IEEE Symposium on Security and Privacy (2024)
4. Apache Software Foundation: Apache teaclave SGX SDK. <https://teaclave.apache.org/> (2024)
5. Arfaoui, G., et al.: Deep attestation: Formally secure attestation across trust domains. In: ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec) (2022)
6. Arnaudov, S., Trach, B., Gregor, F., Knauth, T., Martin, A., Priebe, C., Lind, J., Muthukumaran, D., O’Keeffe, D., Stillwell, M.L., Goltzsche, D., Evers, D., Kapitza, R., Pietzuch, P., Fetzner, C.: SCONE: Secure Linux containers with Intel SGX. In: USENIX Symposium on Operating Systems Design and Implementation (OSDI). pp. 689–703 (2016)
7. Avizienis, A., Laprie, J.C., Randell, B., Landwehr, C.: Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing* **1**(1), 11–33 (2004)
8. Birkholz, H., Thaler, D., Richardson, M., Smith, N., Pan, W.: Remote ATtestation procedures (RATS) architecture. RFC 9334, IETF (Jan 2023)
9. Biryukov, A., Dinu, D., Khovratovich, D.: Argon2: New generation of memory-hard functions for password hashing and other applications. In: IEEE European Symposium on Security and Privacy (EuroS&P). pp. 292–302 (2016)
10. Brandenburger, M., Cachin, C., Lorenz, M., Kapitza, R.: Rollback and forking detection for trusted execution environments using lightweight collective memory. In: IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 157–168 (2017)
11. Bulck, J.V., Minkin, M., Weisse, O., Genkin, D., Kasikci, B., Piessens, F., Silberstein, M., Wenisch, T.F., Yarom, Y., Strackx, R.: Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution. In: USENIX Security Symposium. pp. 991–1008 (2018)
12. Chen, G., Chen, S., Xiao, Y., Zhang, Y., Lin, Z., Lai, T.H.: SgxPectre: Stealing Intel secrets from SGX enclaves via speculative execution. In: IEEE European Symposium on Security and Privacy (EuroS&P). pp. 142–157 (2019)
13. Costan, V., Devadas, S.: Intel SGX explained. *IACR Cryptology ePrint Archive* **2016**, 086 (2016)
14. Crosby, S.A., Wallach, D.S.: Efficient data structures for tamper-evident logging. In: USENIX Security Symposium. pp. 317–334 (2009)
15. Groth, J.: On the size of pairing-based non-interactive arguments. In: *Advances in Cryptology – EUROCRYPT 2016*. pp. 305–326. Springer (2016)
16. Gu, Z., Huang, H., Zhang, J., Su, D., Jamjoom, H., Lamba, A., Pendarakis, D., Molloy, I.: Secure live migration of SGX enclaves on untrusted cloud. In: IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 225–236 (2017)

17. Hussain, M.A., Kanhere, S.S., Jha, S.K.: A survey on online exam proctoring. *Computers & Security* **108**, 102331 (2021)
18. Kaplan, D., Powell, J., Woller, T.: AMD memory encryption. In: AMD White Paper (2020)
19. Knauth, T., Steiner, M., Chakrabarti, S., Li, L., Xing, C., Vij, M.: Integrating remote attestation with transport layer security. In: arXiv preprint arXiv:1801.05863 (2018), rA-TLS
20. Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., Schwarz, M., Yarom, Y.: Spectre attacks: Exploiting speculative execution. In: IEEE Symposium on Security and Privacy (S&P). pp. 1–19 (2019)
21. Kretz, I.D., Parran, C.C., Ramsdell, J.D., Rowe, P.D.: Evidence tampering and chain of custody in layered attestations. In: International Symposium on Principles and Practice of Declarative Programming (PPDP) (2024)
22. Li, M., Zhang, Y., Lin, Z.: CrossLine: Breaking “security-by-crash” based memory isolation in AMD SEV. In: ACM Conference on Computer and Communications Security (CCS). pp. 2937–2950 (2021)
23. McKeen, F., Alexandrovich, I., Berenzon, A., Rozas, C.V., Shafi, H., Shanbhogue, V., Savagaonkar, U.R.: Innovative instructions and software model for isolated execution. In: Workshop on Hardware and Architectural Support for Security and Privacy (HASP). pp. 10:1–10:1 (2013)
24. Nilsson, A., Bideh, P.N., Brorsson, J.: A survey of published attacks on Intel SGX. arXiv preprint arXiv:2006.13598 (2020)
25. Petz, A., Alexander, P.: An infrastructure for faithful execution of remote attestation protocols. In: NASA Formal Methods Symposium. pp. 268–286. Springer (2023)
26. Pinto, S., Santos, N.: Demystifying ARM TrustZone: A comprehensive survey. *ACM Computing Surveys* **51**(6), 130:1–130:36 (2019)
27. Ramsdell, J.D., Rowe, P.D., Alexander, P.: Orchestrating layered attestations. In: International Conference on Principles of Security and Trust (POST). pp. 197–221 (2019)
28. van Schaik, S., Kwong, A., Genkin, D., Yarom, Y.: SGAXe: How SGX fails in practice (2020), extends CacheOut attack to extract SGX attestation keys
29. Schnabl, C., Hugenroth, D., Marino, B., Beresford, A.R.: Attestable audits: Verifiable AI safety benchmarks using trusted execution environments. In: Workshop on Technical AI Governance (TAIG) at ICML (2024)
30. Stefanov, E., van Dijk, M., Shi, E., Chan, T.H.H., Fletcher, C., Ren, L., Yu, X., Devadas, S.: Path ORAM: An extremely simple oblivious RAM protocol. In: ACM Conference on Computer and Communications Security (CCS). pp. 299–310 (2013)
31. Strackx, R., Piessens, F.: Ariadne: A minimal approach to state continuity. In: USENIX Security Symposium. pp. 875–892 (2016)
32. Trivedi, K.S.: Probability and Statistics with Reliability, Queuing, and Computer Science Applications. John Wiley & Sons, 2nd edn. (2002)
33. Tsai, C., Porter, D.E., Vij, M.: Graphene-SGX: A practical library OS for unmodified applications on SGX. In: USENIX ATC (2017)